

A Task Graph Representation for Flexible Hardware/Software Partitioning

Tiago Santos

Univ. of Porto, FEUP and INESC-TEC
Porto, Portugal
tiagolascasas@fe.up.pt

João Bispo

Univ. of Porto, FEUP and INESC-TEC
Porto, Portugal
jbispo@fe.up.pt

João M. P. Cardoso

Univ. of Porto, FEUP and INESC-TEC
Porto, Portugal
jmpc@fe.up.pt

Abstract—Modern FPGA accelerators can be used to accelerate large regions of C/C++ code through High-level Synthesis (HLS), but several challenges persist. One is related to Hardware/Software (Hw/Sw) partitioning, which needs to be more holistic and consider the clustering of code regions based on data communication needs and the potential for code transformations and optimizations. We propose representing a C/C++ application as an extended task-graph with function-level granularity and with a structure and information useful for Hw/Sw partitioning. The proposed approach involves generating a task-graph overlay on the program’s Abstract Syntax Tree (AST) that maps graph operations of task clustering and splitting onto inlining/outlining operations. This maintains the integrity and readability of the original source code while enabling the partitioning of code regions with arbitrary complexity. We evaluate our representation by generating task graphs for the Rosetta and MachSuite benchmark suites, extracting several metrics and parameters from the graphs that can be used to guide novel partitioning and optimization methods.

Index Terms—Task Graph, FPGA, Hardware/Software Partitioning

I. INTRODUCTION

With the advent of modern reconfigurable hardware accelerators and the use of High-level Synthesis (HLS), entire software code regions can be offloaded to an FPGA for significant execution time gains. The increased capacity of these devices makes them capable of accelerating large regions of code. However, typical applications, written in languages such as C/C++, have no clear-cut separation of code that should be offloaded and code that should stay on the host. Finding adequate regions for offloading is, then, of paramount importance in order to take full advantage of current accelerators. This is known as Hardware/Software (Hw/Sw) partitioning, and can be applied using a task graph model of the application [1].

In our work, we consider a task graph with function-level granularity, as we are looking to offload regions consisting of one or more functions [2]. This high granularity presents its own challenges as we need to cluster tasks while preserving/generating a source code representation, as tools in the toolchain require it (e.g., HLS tools). We address the use of the program’s Abstract Syntax Tree (AST), as lowering it down to a lower-level Intermediate Representation (IR) would irreversibly make it more complex to generate code friendly to the tools in the toolchain. Therefore, we propose a task graph formulation that works as an overlay of the program’s AST,

allowing for the clustering/splitting of tasks to be reflected in the underlying AST as function inlining/outlining transformations, as well as to express how the application’s data evolves as it is passed from function to function.

II. TASK GRAPH

Our proposed task graph formulation is a **Directed Acyclical Multigraph**, which is fitting to any AST from an imperative programming language. Each node is either a task connected to other tasks (which represents functions) through their data dependencies or a subgraph, which may contain more tasks, data dependencies, and subgraphs. Subgraphs arise from function calls. Through extensive function outlining of the program’s AST, we guarantee that we only have either functions that call other functions (i.e., have no computations), or functions that only do computations (i.e., call no other functions). However, we allow control flow, such as loops and conditionals, to exist in the former, making it possible to represent a function that is called repeatedly inside a loop or a function that is called conditionally.

This 1:1 mapping between function and task preserves the original source code, and enables transformations performed on the task graph to be reflected in the AST, e.g., merging two tasks in the task graph is reflected in the creation of a new function in the AST consisting of the inlined code of the two original functions. This allows the arbitrary clustering of tasks, as well as to increase/decrease the granularity of the task graph as needed, as every graph operation is reflected as one or more code transformations on the AST (function creation, deletion, inlining and outlining). Generating valid and readable source code from this representation is straightforward, as it can be obtained directly from the AST.

Finally, the graph’s edges represent the communication of data between tasks, with each edge holding aggregated data. These aggregated data items are annotated with the datatype and size of the data (as far as it can be statically determined by our compiler). Order constraints not expressed by data dependencies (e.g., ensuring that two tasks execute one after the other) are expressed using special edges. Furthermore, the conditional execution of tasks is represented by boolean edges.

From an application’s task graph, our compiler extracts several metrics and properties. We highlight herein some that are of particular importance:

TABLE I
METRICS GATHERED FROM THE TASK GRAPHS OF THE SELECTED
BENCHMARKS

Benchmark	#Tasks	#Edges	#SG	Avg. PL	%PCR
MachSuite-backprop	23	130	4	1.22	22.3%
MachSuite-fft-transpose	21	121	5	1	19.8%
MachSuite-kmp	4	20	1	1.5	10.0%
MachSuite-sort-merge	5	28	1	4	17.9%
MachSuite-sort-radix	15	46	2	3.5	21.7%
Rosetta-3d-rendering	23	96	5	1.52	11.5%
Rosetta-digit-recognition	12	45	3	1	15.6%
Rosetta-face-detection	26	219	7	1.62	6.8%
Rosetta-optical-flow	9	84	1	8	0.0%
Rosetta-spam-filter	6	29	1	1	10.3%

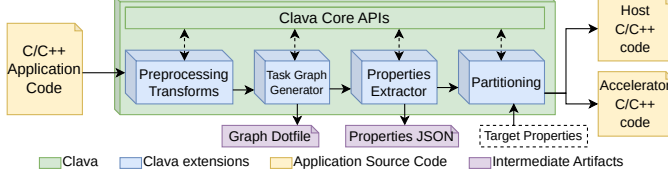


Fig. 1. Task graph creation pipeline and its inputs, as an extension of Clava

- **No-task Calls Histogram:** some function calls (e.g., to `<math.h>` functions) do not lead to the creation of a task, as they can have target-specific implementations and act more as operators;
- **Data Paths:** for each data item in a task, an analysis goes through the task graph and finds the path to the task that first creates the data, or the path to the last task that uses the data;
- **Subgraph Level of Parallelism:** for any subgraph, the compiler shows the relative level of parallelism of its tasks by calculating $|V| \div CP_{len}$, where $|V|$ is the number of tasks in the subgraph and CP_{len} is the length of its critical path;
- **Producer-Consumer Relationships:** for any pair of tasks (T_{source}, T_{target}) where T_{source} communicates data to T_{target} , the compiler estimates whether the communication follows a producer-consumer relationship, i.e., if data written in T_{source} is read in T_{target} . This is useful in determining clusters of tasks that represent a dataflow region, which may be relevant in the context of offloading, as they can be explored by HLS.

III. EXPERIMENTAL RESULTS

To show the versatility of our task graph representation, we select two C/C++ benchmark suites, Rosetta [3] and MachSuite [4], as a source of applications to evaluate the task graph generation of our compiler. Both benchmark suites have been used to evaluate FPGA optimizations and partitioning schemes of source code through HLS, which aligns with the purpose of our task graph. Furthermore, all Rosetta benchmarks and some from MachSuite have multiple functions and complex control flow with call graphs with more than one level of depth, which makes them ideal for the function-level hierarchical granularity of our task graph definition. The

MachSuite benchmarks consisting of a single function are not considered. The task graphs are generated using extensions of the Clava C/C++ to C/C++ source-to-source compiler [5], which also perform the necessary AST transformations (e.g., function inlining/outlining) and preprocessing steps. These extensions are shown in Fig. 1.

Table I shows, for each selected benchmark, the number of tasks, communication edges and subgraphs (#SG) present in its task graph representation. The number of tasks is from 4 to 26, in the *kmp* and *face-detection* benchmark, respectively. On a Hw/Sw Partitioning context, this would mean that our largest search space would have 2^{26} solutions for this level of granularity, if we assume a single host and accelerator. This is an interesting middle point, as it allows us to further change the granularity of a subset of tasks: some tasks could be merged into one, reducing the complexity of the graph without changing its granularity, or a single task could be further split into multiple tasks. Regarding the level of functional parallelism (Avg. PL), some benchmarks show low values. This information is useful, as decomposing the tasks in these highly sequential sections into smaller tasks could help expose parallelism. Lastly, the percentage of communication edges that show a producer-consumer relationship (%PCR) shows that these exist in all but one benchmark, which can inform a partitioning policy centered around dataflow regions.

IV. CONCLUSION

This paper briefly presented our work for representing a C/C++ application, at the AST level, as a function-level task graph bearing in mind Hw/Sw partitioning. Ongoing work focuses on using this representation in the context of researching Hw/Sw partitioning techniques targeting CPU-FPGA systems, where the clusters of tasks for the FPGA are created across data communication connections, while also considering code optimizations enabled by merging tasks. This representation can be used for other scenarios with other accelerator types, or even outside the context of Hw/Sw partitioning.

ACKNOWLEDGMENT

Santos was supported by the Fundação para a Ciência e Tecnologia (FCT) PhD scholarship "2021.07324.BD".

REFERENCES

- [1] N. Hou, X. Yan, and F. He, "A survey on partitioning models, solution algorithms and algorithm parallelization for hardware/software co-design," *Des. Autom. Embedded Syst.*, vol. 23, p. 57–77, jun 2019.
- [2] T. Santos, J. Bispo, and J. M. P. Cardoso, "Cpu-fpga holistic source-to-source compilation approach for partitioning and optimizing c/c++ applications," in *Proc. Int'l Conference on Parallel Architectures and Compilation Techniques (PACT'2023)*, ACM, 2023.
- [3] Y. Zhou, U. Gupta, S. Dai, R. Zhao, N. Srivastava, H. Jin, J. Featherston, Y.-H. Lai, G. Liu, G. A. Velasquez, W. Wang, and Z. Zhang, "Rosetta: A Realistic High-Level Synthesis Benchmark Suite for Software-Programmable FPGAs," *Int'l Symp. on Field-Programmable Gate Arrays (FPGA)*, Feb 2018.
- [4] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "Machsuite: Benchmarks for accelerator design and customized architectures," in *IEEE Int'l Symposium on Workload Characterization (IISWC'2014)*, pp. 110–119, 2014.
- [5] J. Bispo and J. M. Cardoso, "Clava: C/c++ source-to-source compilation using lara," *SoftwareX*, vol. 12, p. 100565, 2020.